



Advanced RL Algorithms for Agents

Improving how agents learn from experience

Carnegie Mellon University
Language Technologies Institute

Graham Neubig
Language Technologies Institute

RL lecture map

RL basics

Sample attempts →
score them → update
the policy



Today

Useful feedback
Less waiting
Stable updates
Reliable rewards
Learning from a
teacher



Next lecture

Memory, parallelism,
and execution at scale

An RL task: fixing a bug

We'll train a coding agent to fix bugs, using tests to judge each attempt.

Prompt

Fix the retry setting: zero should disable retries, but the code returns three. Preserve the other cases.

Initial state: buggy code

```
def retry_count(config):  
    return config.get("retries") or 3
```

Verifier: tests

Input → expected output

Missing → 3

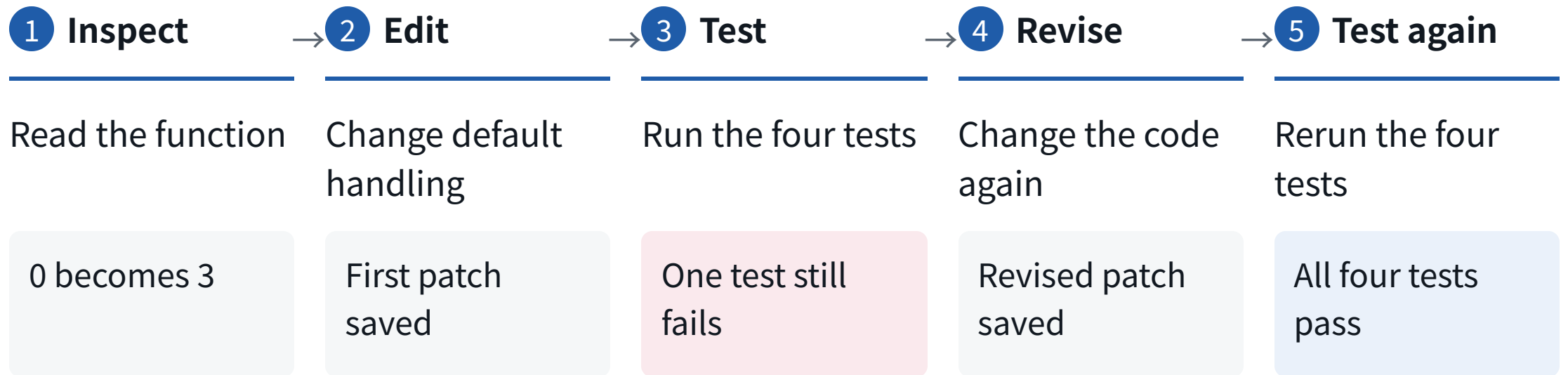
None → 3

0 → 0

2 → 2

A successful trajectory

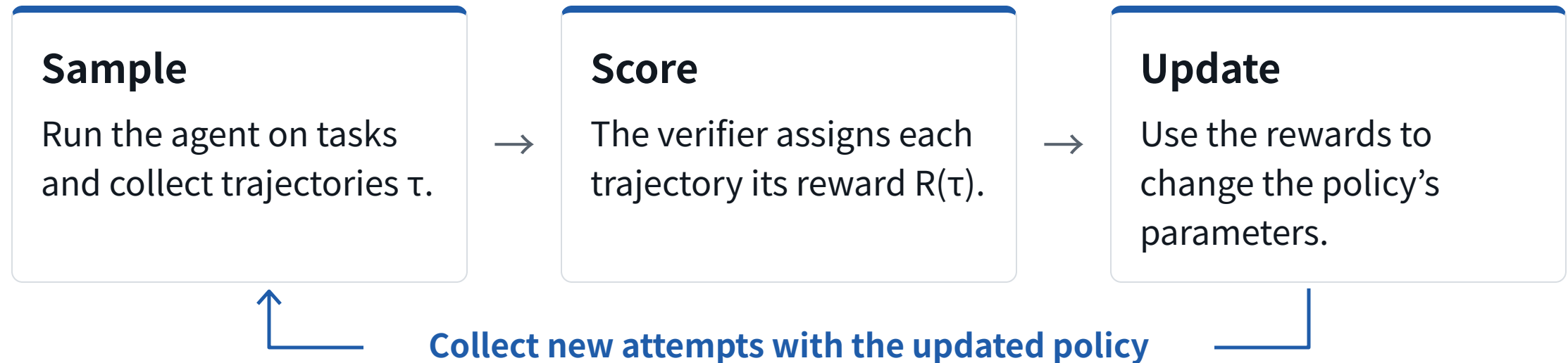
A trajectory τ is one attempt: the agent's actions and the observations they produce.



Final reward: $R(\tau) = 1$ if all four tests pass; 0 otherwise.

The RL training loop

Training repeats the loop to increase average reward over many attempts.



x is the task (initial state and prompt); θ is the policy's parameters.
 $p_{\theta}(\tau \mid x)$ is the probability of trajectory τ given task x .

Goal $J(\theta)$: maximize average reward on task x

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\cdot \mid x)}[R(\tau)]$$

Reward and the policy update

Plain REINFORCE uses the final reward to weight every action in an attempt.

Successful attempt

All four tests pass: $R(\tau) = 1$.

Increase the likelihood of its actions.

Failed attempt

At least one test fails: $R(\tau) = 0$.

This attempt contributes no update.

At step t , action a_t follows history h_t : the task, earlier actions, and observations.

$\pi_\theta(a_t | h_t)$ is the policy's probability of that action given the history.

Loss to minimize for one sampled trajectory

$$\mathcal{L}_{PG} = -R(\tau) \sum_t \log \pi_\theta(a_t | h_t)$$

How can a failed attempt still provide a learning signal?

Credit assignment and dense feedback

Which actions deserve credit?

Comparing trajectories on the same task

Sample four attempts at the same bug-fix task, then compare their rewards.

Attempt	Final tests	Reward $R(\tau)$	Update weight: reward – baseline
1	All four pass	1	$1 - 0.25 = +0.75$
2	At least one fails	0	$0 - 0.25 = -0.25$
3	At least one fails	0	$0 - 0.25 = -0.25$
4	At least one fails	0	$0 - 0.25 = -0.25$

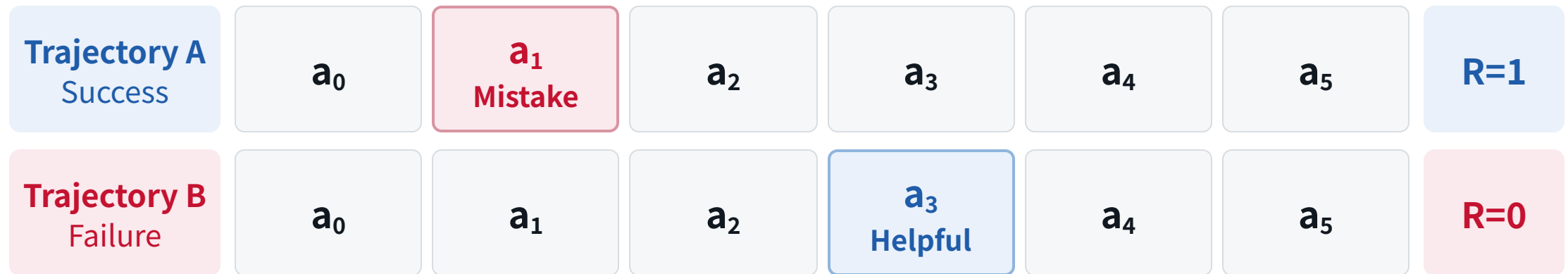
Use the group average as a baseline:
 $(1 + 0 + 0 + 0) / 4 = 0.25$.

Even failed attempts now contribute: their actions get a negative weight.

Rewards for individual turns

A successful trajectory can contain a mistake; a failed trajectory can contain a helpful turn.

Two illustrative trajectories on the same task. Each action $a_0, \dots, a_5$ represents one agent turn.



The group weight is the same for every turn in a trajectory.

Turn-level rewards could tell us which turns to reinforce within each trajectory.

Reward-to-go from one trajectory

With only a final reward, A receives zero at each step until the task succeeds.

History before	h_0	h_1	h_2	h_3	h_4	h_5
Action	$\mathbf{a_0}$	$\mathbf{a_1}$	$\mathbf{a_2}$	$\mathbf{a_3}$	$\mathbf{a_4}$	$\mathbf{a_5}$
Reward after	$r_1 = 0$	$r_2 = 0$	$r_3 = 0$	$r_4 = 0$	$r_5 = 0$	$r_6 = 1$

Starting just before a_3 , how much reward remains on this trajectory?

Add the three remaining rewards, counting each equally

$$0 + 0 + 1 = 1$$

This sum is the **reward-to-go R_3** : the rewards received after h_3 .

Credit along two trajectories

B receives zero reward at every step. Compare its reward-to-go with A's.

Trajectory A Success	a_0 $R_0=1$	a_1 $R_1=1$	a_2 $R_2=1$	a_3 $R_3=1$	a_4 $R_4=1$	a_5 $R_5=1$	$R=1$
Trajectory B Failure	a_0 $R_0=0$	a_1 $R_1=0$	a_2 $R_2=0$	a_3 $R_3=0$	a_4 $R_4=0$	a_5 $R_5=0$	$R=0$

Add the remaining rewards: at every action, $R_t = 1$ on A and $R_t = 0$ on B.

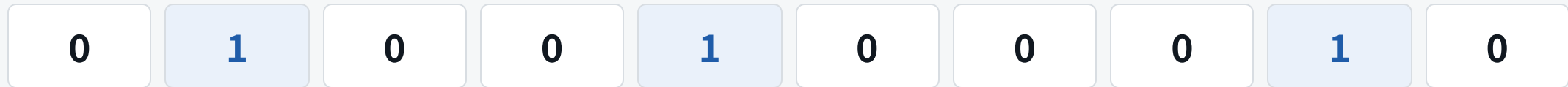
How can we estimate which turns helped when we only observe the final reward?

Expected reward-to-go from a history

Imagine continuing ten times from B's history h_3 , using the same rollout policy μ .

Same h_3 Ten possible continuations $\rightarrow$

Reward-to-go from each continuation (illustrative)



Three continuations have $R_3 = 1$. The average is $3 \div 10 = 0.3$.

Value: the average over all possible continuations under μ

$$V^\mu(h_3) = \mathbb{E}_\mu[R_3 \mid h_3]$$

The bar means “given h_3 .” Ten samples give an estimate of this average.

Learning the value function

Train a critic to predict reward-to-go from the history, using squared error.

Its prediction is $V_\phi(\mathbf{h})$, with learned parameters ϕ . Reuse the ten outcomes at h_3 : three 1s and seven 0s.

If $V_\phi(h_3) = 0.6$, its average squared error is

$$\frac{3(0.6 - 1)^2 + 7(0.6 - 0)^2}{10} = 0.30$$

The best prediction for these samples is their mean, **0.3** (squared error **0.21**).

Minimize over ϕ ; E_t averages histories from sampled rollouts

$$\mathcal{L}_V(\phi) = \mathbb{E}_t \left[(V_\phi(h_t) - R_t)^2 \right]$$

Modeling the value function

A critic can use its own network or read the policy's hidden representation.

Separate value network

History h_t



Value network ϕ

Own body and output layer



Prediction $V_\phi(h_t)$

Value loss trains the whole value network.
Example: InstructGPT.

Value head on policy body

History h_t



Policy body (part of θ)

↓ **stop-gradient**

Linear value head ϕ



Prediction $V_\phi(h_t)$

Value loss trains only the head.
Example: MIXER (Ranzato et al.).

Stop-gradient blocks the value loss from updating the policy body. The policy loss still trains it.

One-step change in value

Here, rewards are zero. Compare the critic's value before and after each action.

Trajectory B · final three actions

a₃

Before	$V_{\phi}(h_3) = 0.3$
Reward	$r_4 = 0$
Next	$V_{\phi}(h_4) = 0.6$

$$0 + 0.6 - 0.3 = \mathbf{+0.3}$$

a₄

Before	$V_{\phi}(h_4) = 0.6$
Reward	$r_5 = 0$
Next	$V_{\phi}(h_5) = 0.4$

$$0 + 0.4 - 0.6 = \mathbf{-0.2}$$

a₅

Before	$V_{\phi}(h_5) = 0.4$
Reward	$r_6 = 0$
Next	$V_{\phi}(h_6) = 0.0$

$$0 + 0.0 - 0.4 = \mathbf{-0.4}$$

B ends after a₅, so no future reward remains at h₆: $V_{\phi}(h_6) = 0$.

Temporal-difference (TD) error, with no reward discounting

$$\delta_t = r_{t+1} + V_{\phi}(h_{t+1}) - V_{\phi}(h_t)$$

Generalized advantage estimation

How should later errors affect the credit given to B's a_3 ?

Trajectory B · final three actions

a_3

$\delta_3 = +0.3$

Weight: 1

a_4

$\delta_4 = -0.2$

Weight: 0.5

a_5

$\delta_5 = -0.4$

Weight: 0.25

Choose $\lambda = 0.5$: each later error gets half the weight of the previous one.

Estimated advantage at a_3 : combine the three errors

$$\hat{A}_3 = 0.3 + 0.5(-0.2) + 0.25(-0.4) = 0.1$$

GAE repeats this backward: $\hat{A}_t = \delta_t + \lambda \hat{A}_{t+1}$. At the end, no later errors remain.

A surrogate: learning value from annotations

A process reward model (PRM) learns step correctness as a surrogate for value.

The denominator of a fraction is 7 less than 3 times the numerator. If the fraction is equivalent to $2/5$, what is the numerator of the fraction? (Answer:)

   Let's call the numerator x .

   So the denominator is $3x-7$.

   We know that $x/(3x-7) = 2/5$.

   So $5x = 2(3x-7)$.

   $5x = 6x - 14$.

   So $x = 7$.

Here, $5x = 6x - 14$ implies $x = 14$. The annotator marks “ $x = 7$ ” as incorrect.

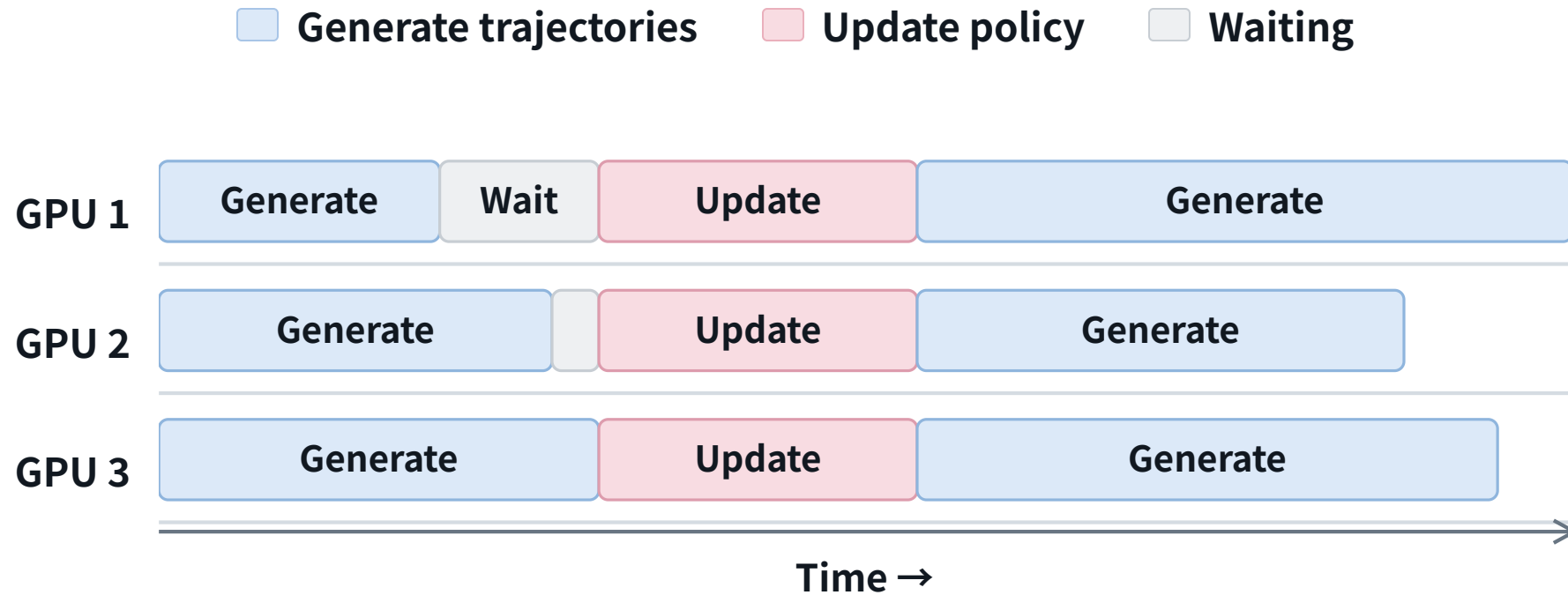
[Let's Verify Step by Step · Lightman et al., 2023 · Figure 1 ↗](#)



Synchronous and asynchronous RL

Waiting in synchronous RL

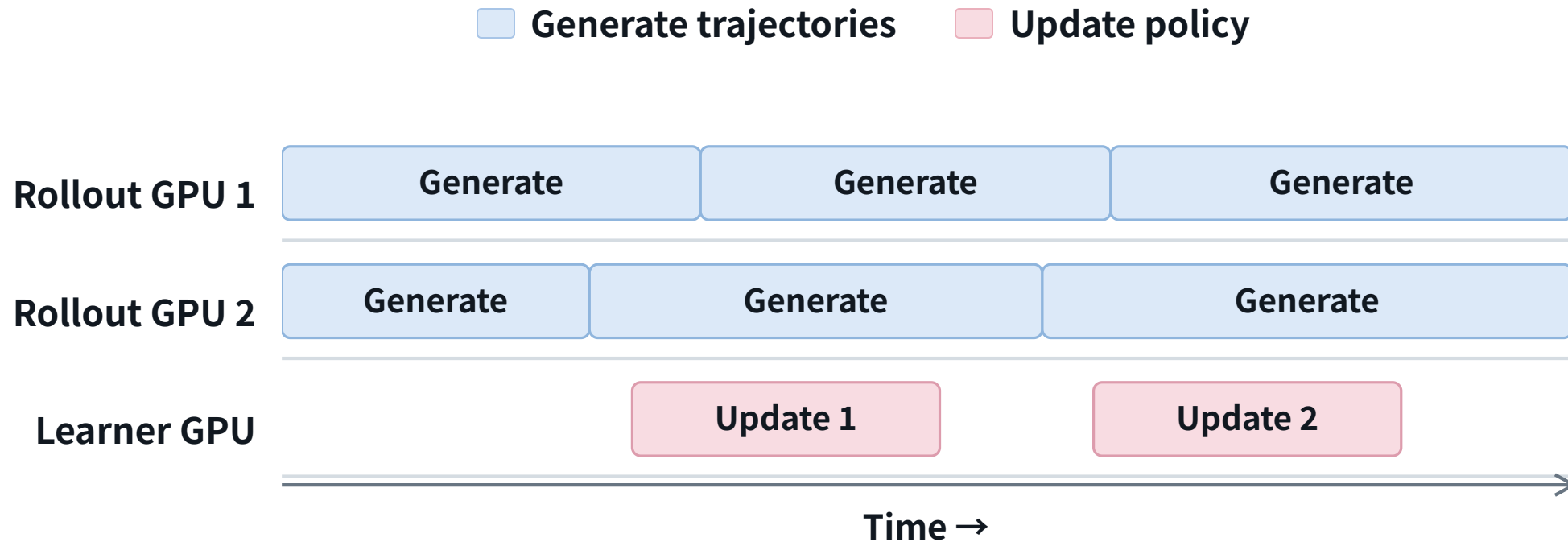
Every trajectory in a batch must finish before the policy update starts.



The next batch starts after the policy update.

Overlapping rollouts and updates

Async RL lets rollout workers keep generating while the learner updates from completed trajectories.

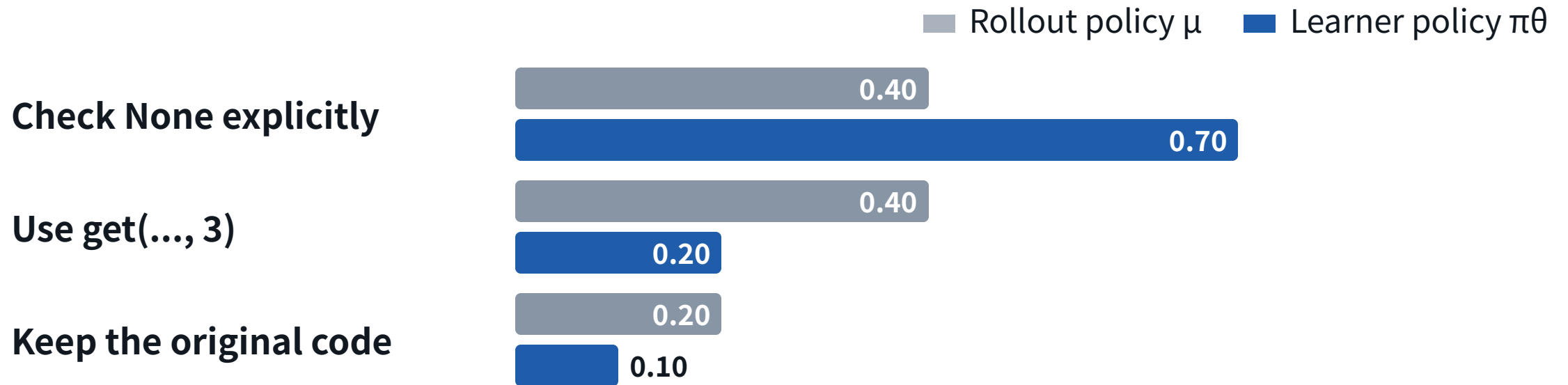


Some completed trajectories now come from an earlier policy version.

Stable policy updates

Rollouts from an older policy

A rollout can finish after the learner has updated to a newer policy.



μ : sampled the recorded edit.

$\pi\theta$: is being updated now.

The rollout is stale: its actions came from μ , while the learner uses $\pi\theta$.

Importance sampling

At a fixed history h , reweight a sampled action a by how likely it is under the current policy.

At a fixed history h

$$\mathbb{E}_{a \sim \pi_\theta} [R(a)] = \mathbb{E}_{a \sim \mu} \left[\frac{\pi_\theta(a | h)}{\mu(a | h)} R(a) \right]$$

Correct edit

Behavior: 0.40

Current: 0.70

Weight: **$0.70 / 0.40 = 1.75$**

Incomplete edit

Behavior: 0.40

Current: 0.20

Weight: **$0.20 / 0.40 = 0.50$**

$R(a)$ is the final reward after taking action a in this example.

Long trajectories and stale data

The whole-trajectory ratio w multiplies local mismatches along an agent interaction.

Action a_t follows history h_t ; environment dynamics are unchanged

$$w(\tau \mid x) = \frac{p_\theta(\tau \mid x)}{p_\mu(\tau \mid x)} = \prod_t \frac{\pi_\theta(a_t \mid h_t)}{\mu(a_t \mid h_t)}$$

1.05

One action ratio



$1.05^{100} \approx 132$

Product along a trajectory

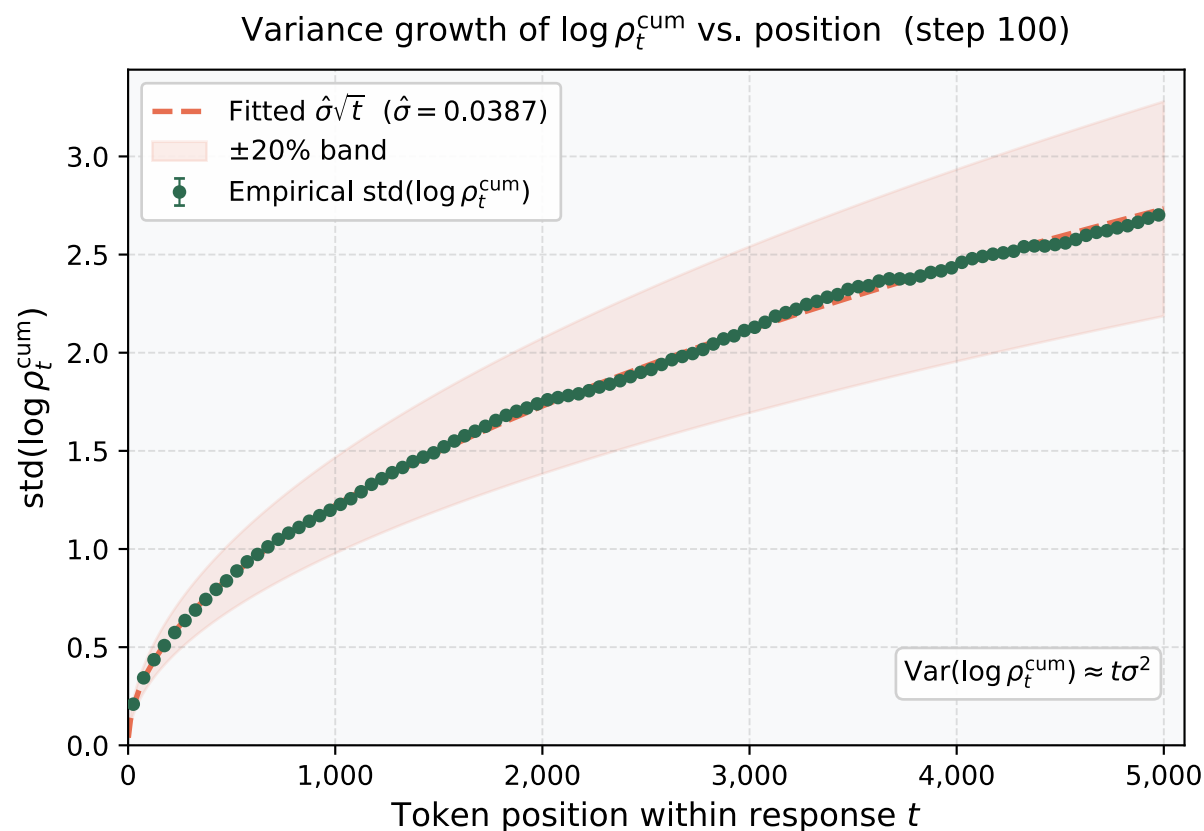


High variance

A few samples can dominate

Mismatch grows along a response

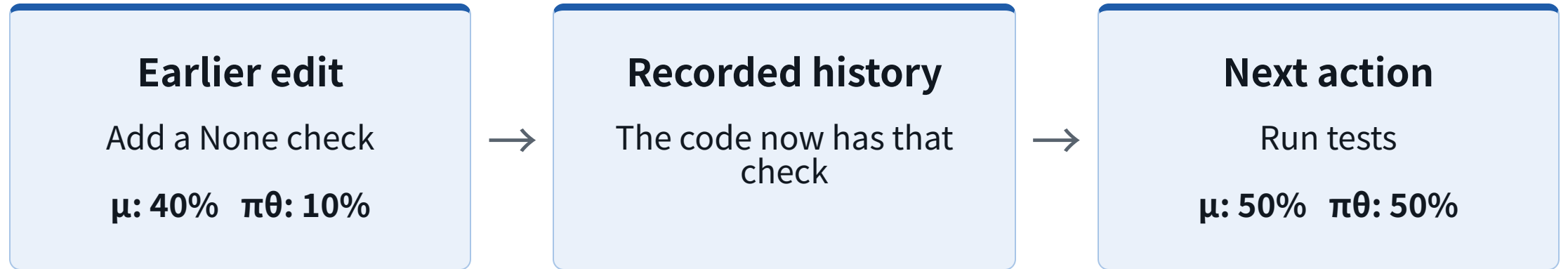
The cumulative ratio ρ_t^{cum} multiplies action ratios through token t .



Its spread grows at later positions in tool-using math rollouts.

Earlier actions change later histories

The current policy may rarely reach a history that the rollout policy visited.



The next-action ratio is **1**, but reaching this history is **0.10 / 0.40 = 0.25** as likely.

Which actions enter the weight?

For a response with T actions, ρ_t is the probability ratio for action t . Methods use different parts of the response.

Method	Recorded actions →	Weight uses
PPO / GRPO	a₀ a₁ a_t a_{t+1} a_{T-1}	Current action
CTPO	a₀ a₁ a_t a_{t+1} a_{T-1}	Actions through t
Full ratio	a₀ a₁ a_t a_{t+1} a_{T-1}	All actions
GSPO	a₀ a₁ a_t a_{t+1} a_{T-1}	All, length-normalized

DAPO and CISPO also use the current-action ratio; their clipping rules differ.

[CTPO · Zhang et al., 2026 · Table 1 ↗](#)

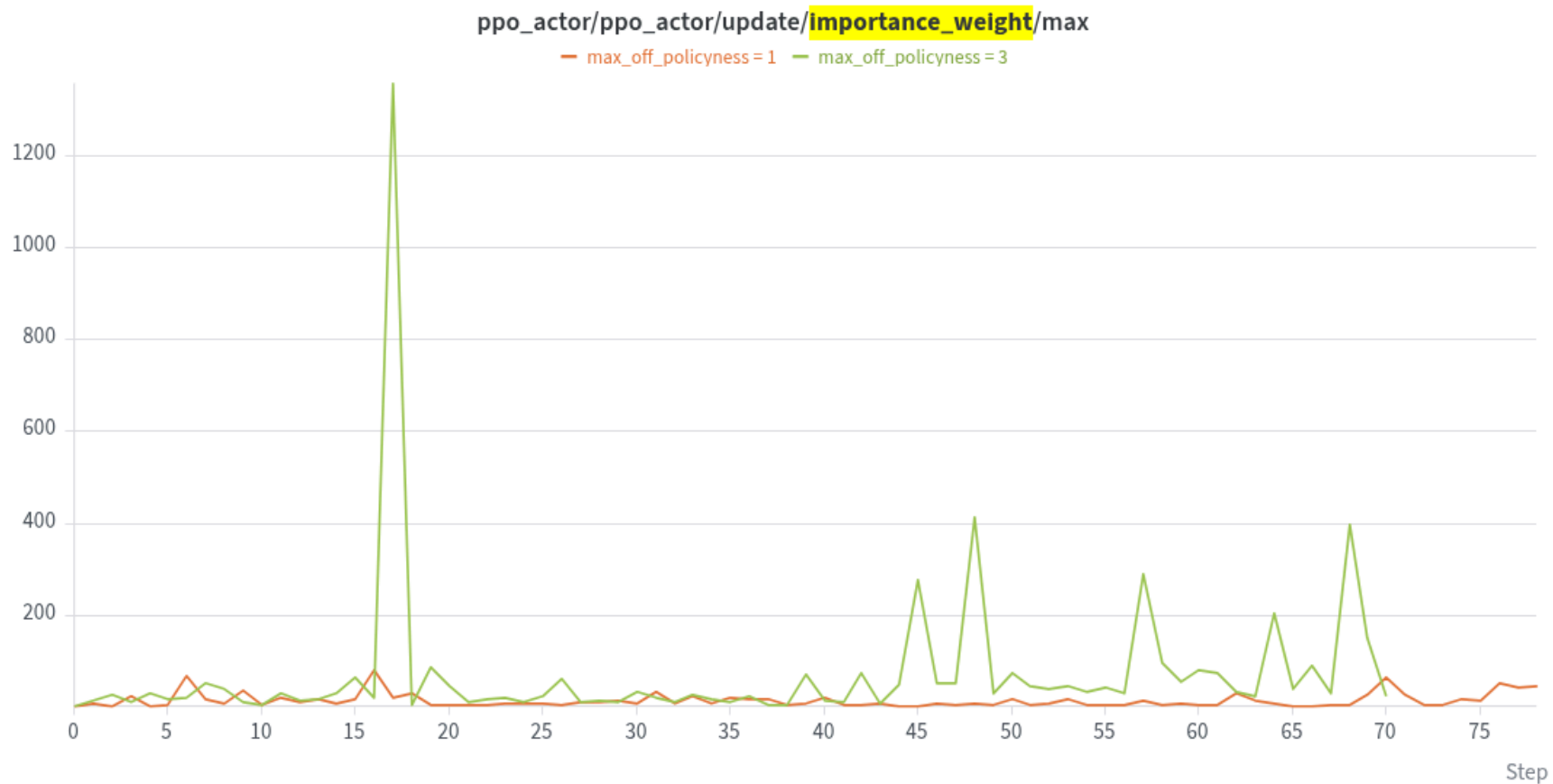
[DAPO · Yu et al., 2025 ↗](#)

[CISPO · MiniMax-M1, 2025 ↗](#)

Clipping policy updates

Why limit an update?

Stale rollouts can get large weights, letting a few samples dominate an update.



Clipping a probability ratio

The clip function replaces a ratio outside the chosen bounds with the nearest bound.

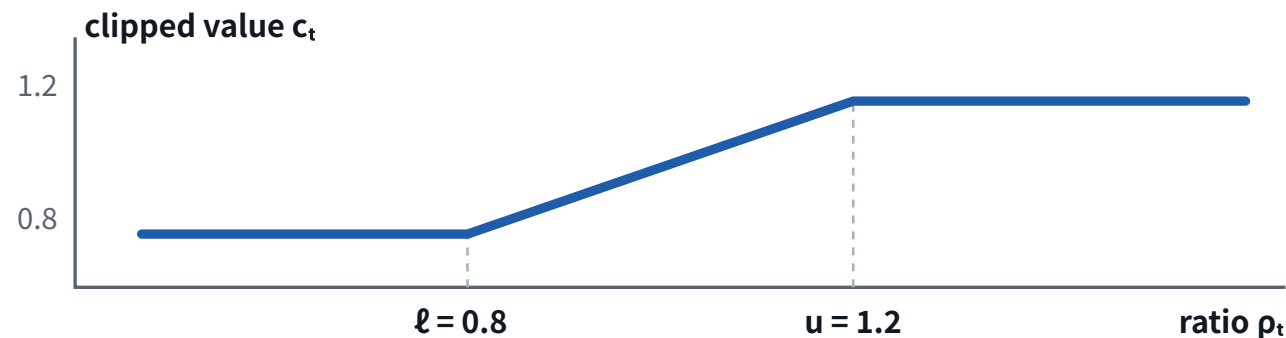
Here each action is a generated token u_t ; $u_{<t}$ is its earlier context.

Ratio for one sampled token

$$\rho_t = \frac{\pi_{\theta}(u_t \mid u_{<t})}{\mu(u_t \mid u_{<t})}$$

Clipped value: ℓ is lower, u is upper

$$c_t = \text{clip}(\rho_t, \ell, u)$$



PPO and CISPO policy objectives

PPO puts c_t inside a minimum; CISPO uses it as a fixed weight.

Shared clipped ratio

$$c_t = \text{clip}(\rho_t, \ell, u)$$

PPO policy objective to maximize

$$J_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, c_t \hat{A}_t \right) \right]$$

CISPO policy objective to maximize

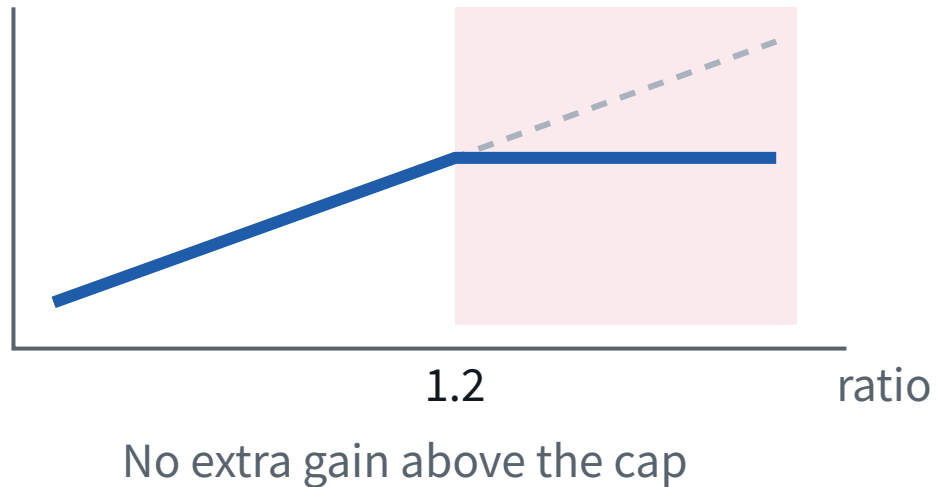
$$J_{\text{CISPO}}(\theta) = \mathbb{E}_t \left[\text{sg}(c_t) \hat{A}_t \log \pi_\theta(u_t \mid u_{<t}) \right]$$

$\mathbb{E}_t$ averages sampled tokens; $\hat{A}_t$ estimates whether a token helped. sg holds c_t fixed for gradients. A loss to minimize is $-J$.

PPO's clipped objective

PPO stops rewarding increases above the cap and decreases below the floor.

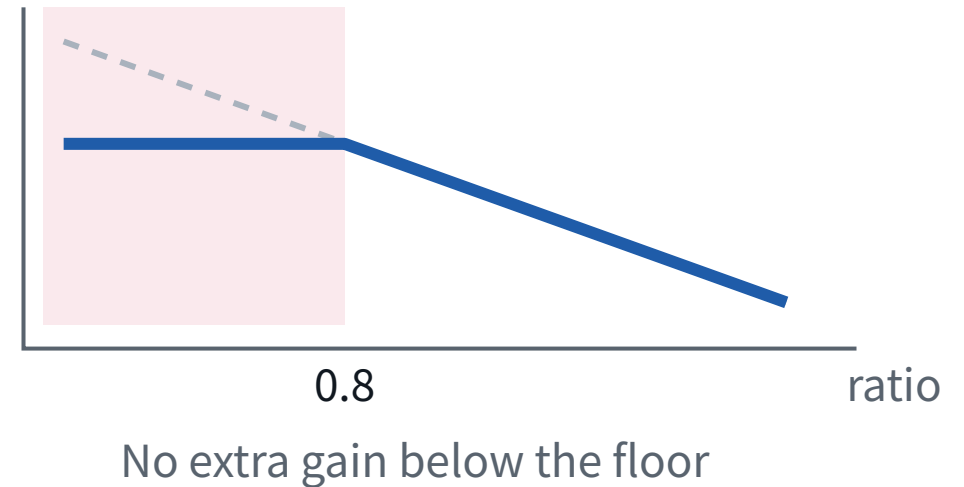
Positive advantage: $A = +1$



For a helpful action, stop rewarding a ratio above $1 + \epsilon$.

In this plot, $\epsilon = 0.2$ is the clipping margin.

Negative advantage: $A = -1$



For an unhelpful action, stop rewarding a ratio below $1 - \epsilon$.

Reference-model KL

A KL penalty discourages the policy from drifting far from a fixed reference model.

Policy objective with a reference penalty

$$\max_{\theta} J_{\text{policy}} - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

Rollout policy μ

Generates the training batch and supplies the denominator in importance ratios.

Reference model π_{ref}

Stays fixed as an anchor while the policy learns.

J_{policy} is the policy objective; D_{KL} measures distribution difference; β sets the penalty strength.

Entropy for exploration

An entropy bonus rewards a broader next-token distribution.

Policy objective with an entropy bonus

$$\max_{\theta} J_{\text{policy}} + \alpha \mathcal{H}(\pi_{\theta})$$

H measures next-token uncertainty; α sets the bonus strength.

- A broader distribution can help the agent try other actions.
- It does not guarantee that those actions will be useful.

A summary of popular algorithms

Compare how each method assigns credit, weights sampled tokens, and limits updates.

Algorithm	Weight from reward	Learned critic?	Importance ratio	Clipping
REINFORCE	Reward-to-go R_t	No	None: fresh rollouts	None
PPO	GAE from a critic	Yes	Current token ρ_t	Minimum of raw and clipped terms
GRPO	Group comparison	No	Current token ρ_t	Same as PPO
CISPO	Group comparison	No	Current token ρ_t	Clipped ratio as a fixed weight
GSPO	Group comparison	No	Whole response, length-normalized	PPO minimum, on the response ratio
DAPO	Group comparison	No	Current token ρ_t	PPO minimum; higher upper bound

Group comparison: subtract the group's mean reward, then divide by its reward spread.

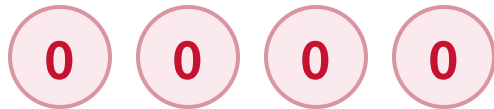


Task selection and reward reliability

Uninformative rollout groups

Sample four trajectories per task. Each group below shows their final rewards.

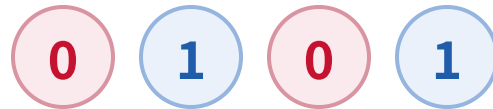
All attempts failed



Mean reward: 0
Subtract the mean:

0, 0, 0, 0

Mixed outcomes



Mean reward: 0.5
Subtract the mean:

-0.5, +0.5, -0.5, +0.5

All attempts passed



Mean reward: 1
Subtract the mean:

0, 0, 0, 0

Recall group advantages: subtract the group's mean reward. Equal rewards all become zero.

An all-zero group gives no outcome comparison. Why did every trajectory fail?

Why did every trajectory fail?

Four failed trajectories give the same rewards: 0, 0, 0, 0. Inspect what happened before choosing a remedy.

Verifier quality

A valid solution might be rejected.

Check: the requirements and failed tests.

Current capability

The model may not yet be able to solve the task.

Check: whether stronger-model demonstrations succeed.

Lack of exploration

All attempts may repeat the same approach.

Check: whether the trajectories differ.

False negatives in the verifier

Return to the bug-fix task. This candidate produces every required output.

Missing → 3 ✓

None → 3 ✓

0 → 0 ✓

2 → 2 ✓

Candidate behavior

All four cases are correct.

Its implementation uses different source code.

Verifier rule

Require the exact phrase

`value is None`.

The candidate lacks it: **reject**.

This is a **false negative**: a valid solution is rejected. Check behavior instead of an incidental wording choice.

[Related task/test review: SWE-bench Verified · 2024 ↗](#)

Result of false negatives

Reward noise

Fitting to spurious changes in the output.

Equivalent solutions receive different rewards because of wording or implementation details.

Benchmark saturation

Scores can flatten below 100% when the evaluator rejects valid solutions.

Benchmark	Plateau / limit	Evidence from the audit
SWE-bench Verified	≈81%	Progress slowed near 80.9%; many remaining failures had flawed tests.
τ-bench Airline	≈70%	Inconsistent tasks limited achievable scores.
τ-bench Retail	≈92%	Annotation errors limited achievable scores.

Historical versions, audited in 2025–26. These are reported plateaus or limits; τ-bench has since corrected the tasks.

False positives and reward hacking

Keep the same task requirements, but suppose training tests only the input retries=0.

```
def retry_count(config):  
    return 0
```

Candidate: always return zero.

Training verifier

The one tested case passes.

Reward: **1**

Input	Required output	Candidate output
0	0	0 ✓
Missing / None	3	0 ✗
2	2	0 ✗

A **false positive** rewards an invalid solution. Training can learn to exploit this gap.

Reward hacking in coding agents

A passing score can come from a shortcut that the benchmark was meant to exclude.

Available in the environment		Shortcut
Internet access	→	Find the published solution online.
Git history, including future commits	→	Find and copy the later bug fix.
A model API key	→	Call a different, stronger model for answers or training data.
Access to tests or the test runner	→	Make the tests pass without fixing the bug.

Documented API example: a PostTrainBench agent used a grading key to generate training data, despite an explicit restriction.

Independent checks of progress

The constant-zero patch earned training reward 1, but passed only one of the four required cases.

Training reward

Measures success on the training verifier

Independent success

Check unseen tasks and behavior the training verifier missed

Costs and failures

Inspect tool calls, regressions, and repeated actions

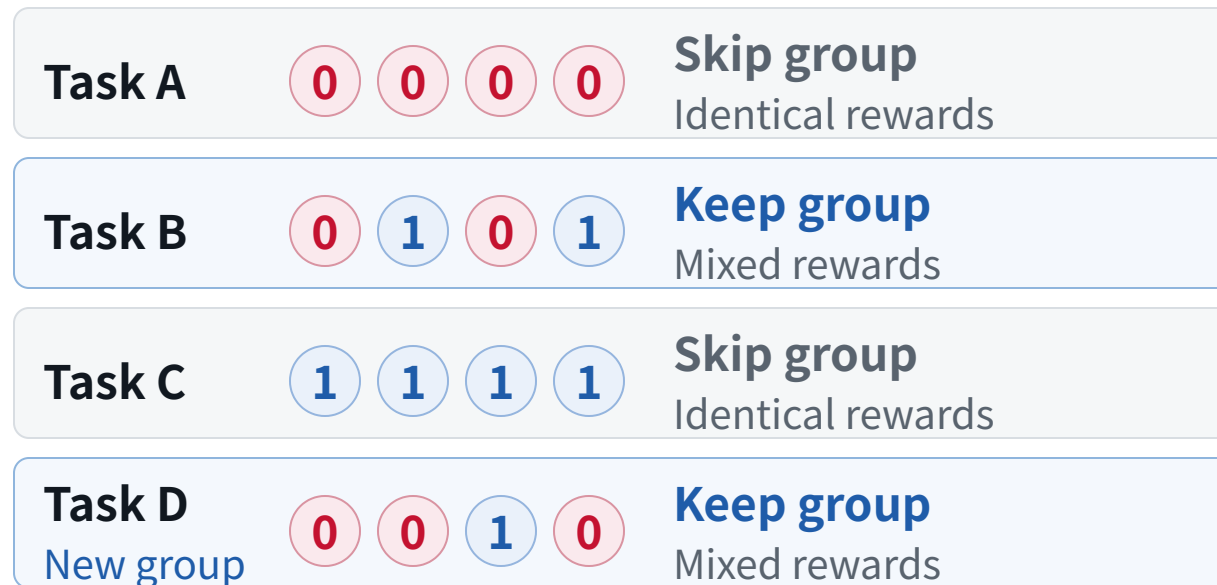
If reward rises without independent improvement, inspect trajectories for shortcuts.

DAPO: dynamic sampling

Identical rewards give zero group advantages. DAPO keeps mixed groups and samples until the batch is full.

Sample four trajectories per task

Each circle is a reward: 0 = failure, 1 = success.



Training batch

Example target: 2 groups

2 / 2 groups collected

Task B: 0, 1, 0, 1

Task D: 0, 0, 1, 0

Batch full → update the policy

Keep every trajectory in a mixed group, including its failures.

Partial and auxiliary rewards

Partial credit is an auxiliary reward: extra feedback for progress before full success.

Illustrative task: pass all 4 checks. Add 0.25 reward per check passed.

Four trajectories on the same task	τ_1	τ_2	τ_3	τ_4
Checks passed	0 / 4	1 / 4	2 / 4	1 / 4
Success reward	0	0	0	0
Subtract the mean: 0	0	0	0	0
Success + partial reward	0	0.25	0.50	0.25
Subtract the mean: 0.25	-0.25	0	+0.25	0

Different partial scores can reduce zero-advantage groups and batches.

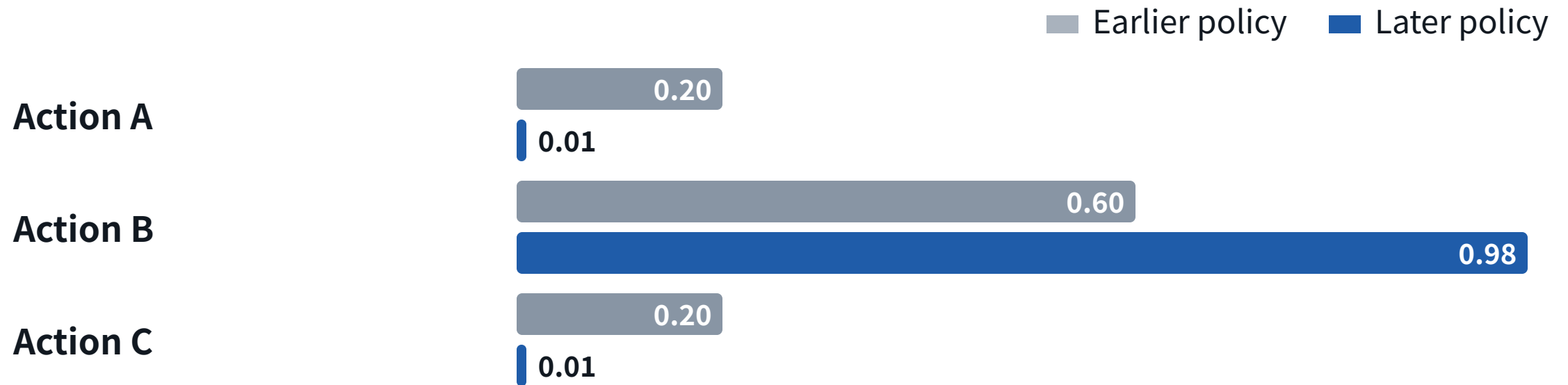
Risks of auxiliary rewards

The model may learn to increase the added reward without improving the task.

Intended behavior	What the reward favored	Observed behavior
Useful tool use	A bug gave credit for superficial web-tool calls.	GPT-5.1 used the browser as a calculator and acted as though it had searched.
A playful “Nerdy” persona	Metaphors with “goblin” or “gremlin” received higher scores.	Creature metaphors became more common, including without the persona prompt.

Exploration collapse

At one history, training changes the probabilities of three possible actions.



- Almost every sample now chooses B, so alternatives are rarely tried.
- An entropy bonus encourages alternatives; check whether trajectories differ.

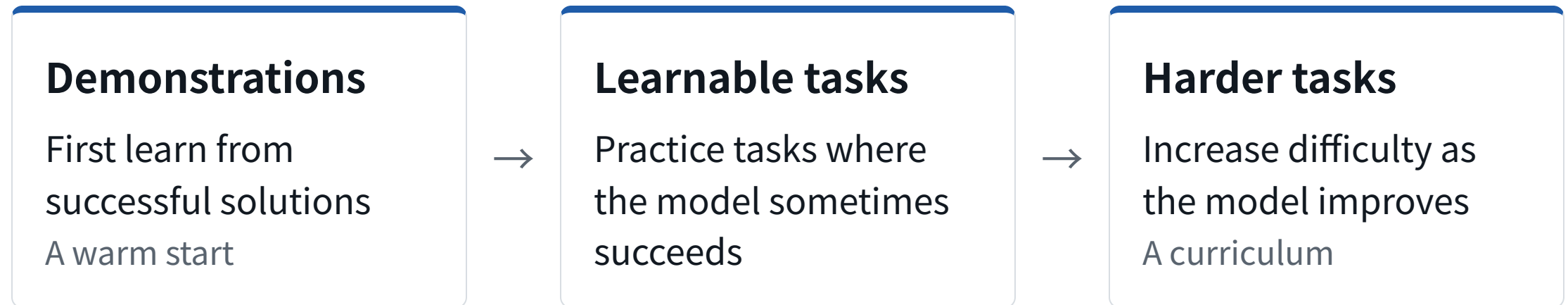


Learning from a teacher

Distillation: match a teacher's action probabilities

Curricula and warm starts

When the student rarely succeeds, a teacher can provide successful trajectories to learn from.

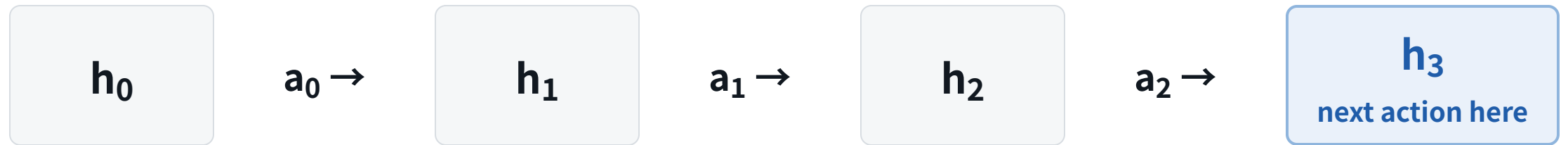


Keep earlier tasks in the mix, and use a fixed evaluation set to measure improvement.

Distillation

Train a student to match a teacher's action probabilities on a fixed set of trajectories.

Saved trajectory from the teacher



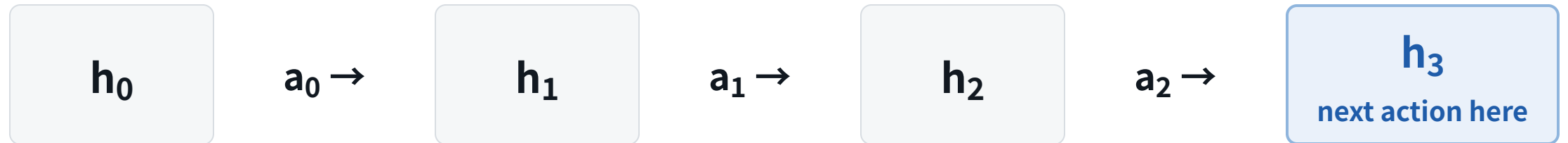
At this saved history, the teacher assigns **80% to a_3** and **20% to the other action**. Call this distribution $q(\cdot | h_3)$.

When the student acts, its own mistakes can lead to histories missing from this training set.

On-policy distillation

Let the student generate the training histories, then ask the teacher for targets at those histories.

Student rollout on trajectory B



Example teacher targets at this student history: **80% for a_3 and 20% for the other action.**

Train the student to match these probabilities. “On-policy” means the histories came from the student's own rollouts.

A dense distillation objective

At the same h_3 , suppose the student assigns equal probability to the two actions.

Next-action probabilities	a_3	Other action
Student π_θ	0.5	0.5
Teacher q	0.8	0.2

Reverse KL measures probability mismatch (natural logarithms)

$$0.5 \log(0.5/0.8) + 0.5 \log(0.5/0.2) \approx 0.223$$

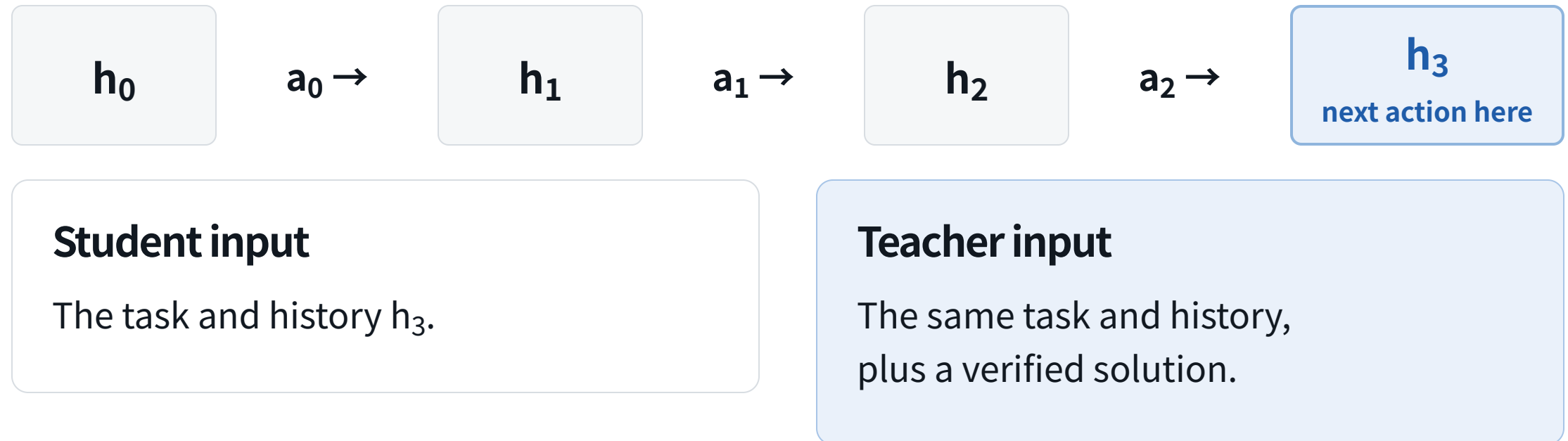
Average this KL mismatch over student histories (distribution d_μ), then minimize:

$$L_{\text{OPD}} = \mathbb{E}_{h \sim d_\mu} [D_{\text{KL}}(\pi_\theta(\cdot \mid h) \parallel q(\cdot \mid h))]$$

On-policy self-distillation

A verified solution can help a teacher give better targets at the student's own history.

Student rollout on trajectory B



OPSD uses a frozen copy of the starting model as teacher. Its next-action probabilities train the student as before.



Conclusion

RL takeaways

Useful feedback

Use group comparisons, a critic, or process rewards to assign credit.

Stable updates

Know the behavior policy; control how strongly samples change the model.

Reliable rewards

Match task difficulty and verify the behavior you actually want.



Questions?

Next Class: RL Systems

Notation: the rollout

Symbol	Meaning
x, τ, T	Task; sampled trajectory; number of actions
h_t, a_t	History before step t ; action at that step
$u_t, u_{<t}$	Generated token at position t ; earlier context
π_θ, μ	Current policy; policy that generated the rollout
$p_\theta(\tau \mid x), p_\mu(\tau \mid x)$	Probability of the whole trajectory under each policy
$R(\tau), r_{t+1}, R_t$	Final reward; reward after a_t ; reward-to-go from h_t

Notation: learning signals

Symbol	Meaning
$\hat{A}_t$	Estimated advantage: how an action compares with expectation
ρ_t, c_t	One-token policy ratio; its clipped value
$\rho_t^{\text{cum}}, w(\tau x)$	Ratio through token t; ratio for the whole trajectory
$V^\mu(h), V_\phi(h)$	Expected reward-to-go under μ ; the critic's estimate
δ_t	Reward plus next value, minus current value
γ, λ	Reward discount (examples use $\gamma = 1$); weight on later errors in GAE
q, d_μ	Teacher's target; histories collected by the student